INDEX

Object-Oriented Programming (OOP) Notes

Master Table of Contents & Learning Index

Module	Core Topic Area	Key Concepts
PART 1	OOP Fundamentals	Class vs Object , Instantiation , Member Functions , Access Control
PART 2	Constructors & Destructors	Default / Parameterized , Copy Constructor , Shallow vs Deep Copy , RAII
PART 3	Inheritance & Hierarchy	Base/Derived Classes , Access Modes , Diamond Problem , Method Overriding
PART 4	Polymorphism & Abstraction	Function/Operator Overloading , Virtual Functions , Abstract Classes , VTable
PART 5	Memory & Special Keywords	`this` Pointer, Static Members , Friend Functions , Const Correctness
PART 6	Templates , Exceptions & Extended Overloading	Generic Functions / Classes , Try-Catch Mechanics , Friend Binary Operations , Complex Division Derivations

PART 1 — OOP Fundamentals

1.1 Paradigm Overview

- Procedural vs. Object-Oriented Programming (OOP)

- Core benefits: Modularity, Code Reusability, Maintainability, Data Security

1.2 Class vs. Object

- **Class**: Structural blueprint template defining member attributes and functions
- **Object**: Real-world entity manifestation created in memory holding state values

1.3 Instantiation & Memory Allocation

- Declaration vs. Instantiation syntax (e.g., `Student s1;`)
- How stack vs. heap dynamic allocation stores object member variables

1.4 Encapsulation & Access Modifiers

- **Data Hiding** : Preventing direct external access to sensitive class members
- Access Specifiers : `private` , `public` and `protected`
- Direct Access Restrictions (e.g., `s1.name = "John"` throwing compiler error)
- Accessor (Getters) and Mutator (Setters) functions (`setData()` , `getData()`)

1.5 Member Functions & Function Invocation

- Defining member functions inside vs. outside class using Scope Resolution Operator (`::`)
- Function invocation syntax using the dot operator (`s1.setData()`)
- Inline member functions and compiler optimizations

PART 2 — Constructors & Destructors

2.1 Constructor Fundamentals

- Definition, characteristics, and automatic execution upon instantiation
- Default constructors generated by the compiler

2.2 Parameterized & Overloaded Constructors

- Passing arguments during object initialization
- Constructor overloading and signature matching

2.3 Copy Constructor & Copy Initialization

- Syntax and purpose : `Student(const Student &source)`
- When copy constructors are invoked (pass-by-value , object returns)

2.4 Deep Copy vs. Shallow Copy

- Default bitwise dynamic member copy pitfalls (dangling pointers, double free)
- Implementing custom Deep Copy with explicit memory allocation

2.5 Destructors & Resource Management

- Destructor syntax (`~ClassName()`) and execution timing
- Resource Acquisition Is Initialization (RAII) pattern
- Memory leak prevention and explicit cleanup

PART 3 — Inheritance & Class Hierarchies

3.1 Base and Derived Classes

- Understanding Parent (Base) vs. Child (Derived) relationships
- Code reuse and class specialization

3.2 Access Modes in Inheritance

- `public` inheritance mode (preserving public interface)
- `protected` inheritance mode (restricting interface access)
- `private` inheritance mode (implementation detail encapsulation)

3.3 Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance

- Hybrid Inheritance

3.4 Ambiguity & The Diamond Problem

- Multiple inheritance member collision issues
- Virtual Base Classes (`virtual public Base`) to resolve diamond redundancy
- Constructor / Destructor execution order in derived chains

PART 4 — Polymorphism & Abstraction

4.1 Compile-Time Polymorphism (Static Binding)

- Function Overloading rules and name mangling
- Operator Overloading (Unary & Binary operators)

4.2 Run-Time Polymorphism (Dynamic Binding)

- Virtual functions and late binding mechanism (`virtual void show()`)
- Function overriding in derived classes vs. function hiding
- Override specifiers (`override` , `final`)

4.3 Virtual Table (VTable) Mechanics

- How Virtual Pointers (`vptr`) and Virtual Tables (`vtable`) work under the hood
- Memory overhead of dynamic dispatch

4.4 Abstract Classes & Interfaces

- Pure Virtual Functions (`virtual void voidFunc() = 0;`)
- Designing Abstract Base Classes (ABCs)
- Interfacing and contract enforcement

PART 5 — Advanced Language Mechanics & Keywords

5.1 The `this` Pointer

- Implicit self-referencing pointer inside non-static member functions
- Resolving variable scope collisions and supporting method chaining (`return *this;`)

5.2 Static Members

- Static Data Members: Shared memory variables across all instances
- Static Member Functions: Calling functions without instantiating objects (`Class::Function()`)

5.3 Friend Functions & Friend Classes

- Granting private member access to non-member external functions (`friend`)
- Encapsulation trade-offs and controlled tight coupling

5.4 Const Correctness

- Const member functions (`void display() const;`)
- Const objects and immutable state guarantees

PART 6 — Templates & Exception Handling

6.1 Generic Programming with Templates

- Function Templates (`template <typename T>`)
- Class Templates and custom data structure implementations

6.2 Exception Handling Framework

- Error handling flow : `try` , `throw` and `catch` blocks
- Catching standard exceptions (`std::exception`)
- Creating custom user-defined Exception Classes

Extended Operator Overloading & Friend Functions

6.1 Friend Function Architecture for Binary Operations

- **Encapsulation vs. Friend Functions**: Friend functions provide controlled access to private members for external non-member functions without breaking encapsulation principles.
- **Syntax & Rules**: Declared with the `friend` keyword inside the class definition; defined outside without the scope resolution operator (`::`) and without receiving an implicit `this` pointer.
- **Multiplication Friend Function**: Formula signature: `friend Complex mul(const Complex& c1 , const Complex& c2);` where (a + ib)(c + id) = (ac - bd) + i(ad + bc) .

6.2 Binary Operator Overloading Calling Conventions: Implicit vs. Explicit Operands

- The expression `c1 / c2` implicitly translates to the member function called `c1.operator/(c2)`.
- The left operand (`c1`) is passed implicitly via this pointer , while the right operand (`c2`) is passed explicitly as an argument.
- Overloading division (`/`) is necessary because native C++ division operators only support primitive data types , not user-defined complex structures.

6.3 Mathematical Derivations of Complex Division

- **Conjugate Rationalization**: (a + ib) / (c + id) = ((ac + bd) + i(bc - ad)) / (c^2 + d^2)
- **Real Part Formula**: (real * c2.real + imag * c2.imag) / (c2.real^2 + c2.imag^2)
- **Imaginary Part Formula**: (imag * c2.real - real * c2.imag) / (c2.real^2 + c2.imag^2)

😁 PART -1

💻 *Object-Oriented Programming (OOP) Fundamentals*

Comprehensive Notes : Encapsulation Mechanics , Access Control and C++ Implementation Details

---

1. 🏗️ Core Concepts: Class vs. Object

At its foundation, Object-Oriented Programming structures software around data entities and well-defined interfaces rather than isolated logic procedures.

- **Class**: A user-defined blueprint, design template, or type specification (analogous to an un-filled application form or an architectural drawing). Groups related data attributes and behaviors into a single cohesive unit without allocating memory for instance data upon definition.
- **Object**: A concrete instance created from a class blueprint. Represents a real-world entity existing in memory that holds actual data state.
- **Instantiation**: The technical runtime process of creating an instance (object) of a defined class, allocating the necessary memory to store its attributes.

> 💡 Class vs Object at a Glance
> Blueprint (Class): class Student { int roll_no; string name; }; — No memory allocated.
> Instance (Object): Student s1; — Allocates memory for s1 in RAM at runtime.

2. 🛡️ Why OOP & Encapsulation?

Procedural implementations often rely on loose, unassociated variables (such as managing separate arrays or standalone variables for `student_name`, `roll_number`, `gpa`, and `age`). As dynamic applications scale, this loose structure becomes clutter, fragile, and highly error-prone.

Encapsulation solves this structural risk by bundling state attributes (data members) and behavioral logic (member functions) into a unified, self-contained entity. This defensive design pattern hides internal complexity and prevents direct external tampering with sensitive state information.

3. 🔑 Access Modifiers & Data Security

Access modifiers enforce encapsulation by defining explicit visibility boundaries for class members:

Modifier	Accessibility Scope	Primary Purpose
`private`	Accessible **only** within member functions of the same class.	Protects sensitive internal state from unvalidated external modification (e.g., in `main()`).
`public`	Accessible from **anywhere** in the program via an instance.	Exposes public interfaces, controls, getters, and setters to interact with the object.
`protected`	Accessible within the class and derived (inherited) classes.	Maintains encapsulated boundaries across hierarchical inheritance trees.

⚙️ C++ Structural Comparison: `struct` vs. `class`

In C++, both `struct` and `class` mechanisms can contain both data members and member functions. Their sole core difference lies in default access visibility:

- **`struct`**: Default access visibility is **`public`**.
- **`class`**: Default access visibility is **`private`**.

Feature	`struct`	`class`
Default Visibility	**public**	**private**
Primary Usage	Plain Old Data (POD) structures	Encapsulated OOP entities with methods

4. ⚙️ Member Functions & Method Invocation

To manipulate or access private class variables safely from external execution contexts, public member functions act as controlled gateways:

- **Setters / Mutators / Input Functions** (e.g., `setData()`): Receive external inputs, validate data integrity, and update internal private attributes safely.
- **Getters / Accessors / Display Functions** (e.g., `getData()`): Inspect and output object state values without granting raw memory write permissions.
- **Function Invocation**: Executing a member function targeting a distinct object instance utilizing the member access dot (`.`) operator (e.g., `s1.setData()`).

5. 🖥️ Complete C++ Implementation (VS Code Style)

```cpp
// ============================================================
// File: Student.cpp
// Topic: C++ OOP Class Implementation & Encapsulation
// ============================================================

#include <iostream>
#include <string>
using namespace std;

// --- 1. Class Definition ---
class Student {
private:
    // Encapsulated Attributes (Private Data Members)
    int roll_no;
    string name;
    float gpa;
    int age;

public:
    // Member Function: Setter / Mutator
    void setData() {
        cout << "Enter Roll Number: ";
        cin >> roll_no;
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter GPA: ";
        cin >> gpa;
        cout << "Enter Age: ";
        cin >> age;
    }

    // Member Function: Getter / Accessor
    void getData() {
        cout << "\n--- Student Details ---" << endl;
        cout << "Roll Number : " << roll_no << endl;
        cout << "Name        : " << name << endl;
```

```
        cout << "GPA         : " << gpa << endl;
        cout << "Age         : " << age << endl;
    }
};

// --- 2. Main Execution ---
int main() {
    // Instantiation: Creating objects of type Student
    Student s1;
    Student s2;

    // Direct access to private members is ILLEGAL:
    // s1.roll_no = 101; // Compilation Error!

    // Function Invocation: Calling public member functions
    cout << "Enter details for Student 1:\n";
    s1.setData();

    cout << "\nEnter details for Student 2:\n";
    s2.setData();

    // Displaying data via getters
    s1.getData();
    s2.getData();

    return 0;
}
```

6. 📌 Key Exam Summary & Takeaways

- **Class**: A blueprint or custom type definition holding no instance data on its own.
- **Object**: A concrete, allocated memory instance created from a class structure.
- **Encapsulation**: Bundling data attributes and functional behaviors while shielding internal implementation details.
- **Access Control**: Using `private` visibility to protect underlying state and `public` visibility to present functional interfaces.
- **Invocation**: Running an object's implementation routine via dot notation syntax (`object.function()`).

> **📝 Quick Exam Checklist**
>
> **1. Class vs Object:** Blueprint vs memory instance.
>
> **2. Default Access:** Struct defaults to `public`; Class defaults to `private`.
>
> **3. Encapsulation:** Keep variables private, expose public getters/setters.

😍 PART – 2

💻 Object-Oriented Programming : Classes , Objects & Data Encapsulation

notes on core OOP , memory models, access boundaries, and C++ implementation.

💡 At a Glance: Blueprint vs. Instance

- **Class**: The conceptual blueprint or schema (e.g., an empty college examination form template). It defines structure, properties, and constraints without allocating storage for specific entity values.
- **Object**: The concrete instantiation (e.g., a completed form bearing an individual student's actual credentials). An active, memory-allocated manifestation of the class blueprint.
- **Encapsulation**: Bundling state attributes (`name`, `roll_number`) together with the operations that manipulate them, shielding sensitive data behind verified access barriers.

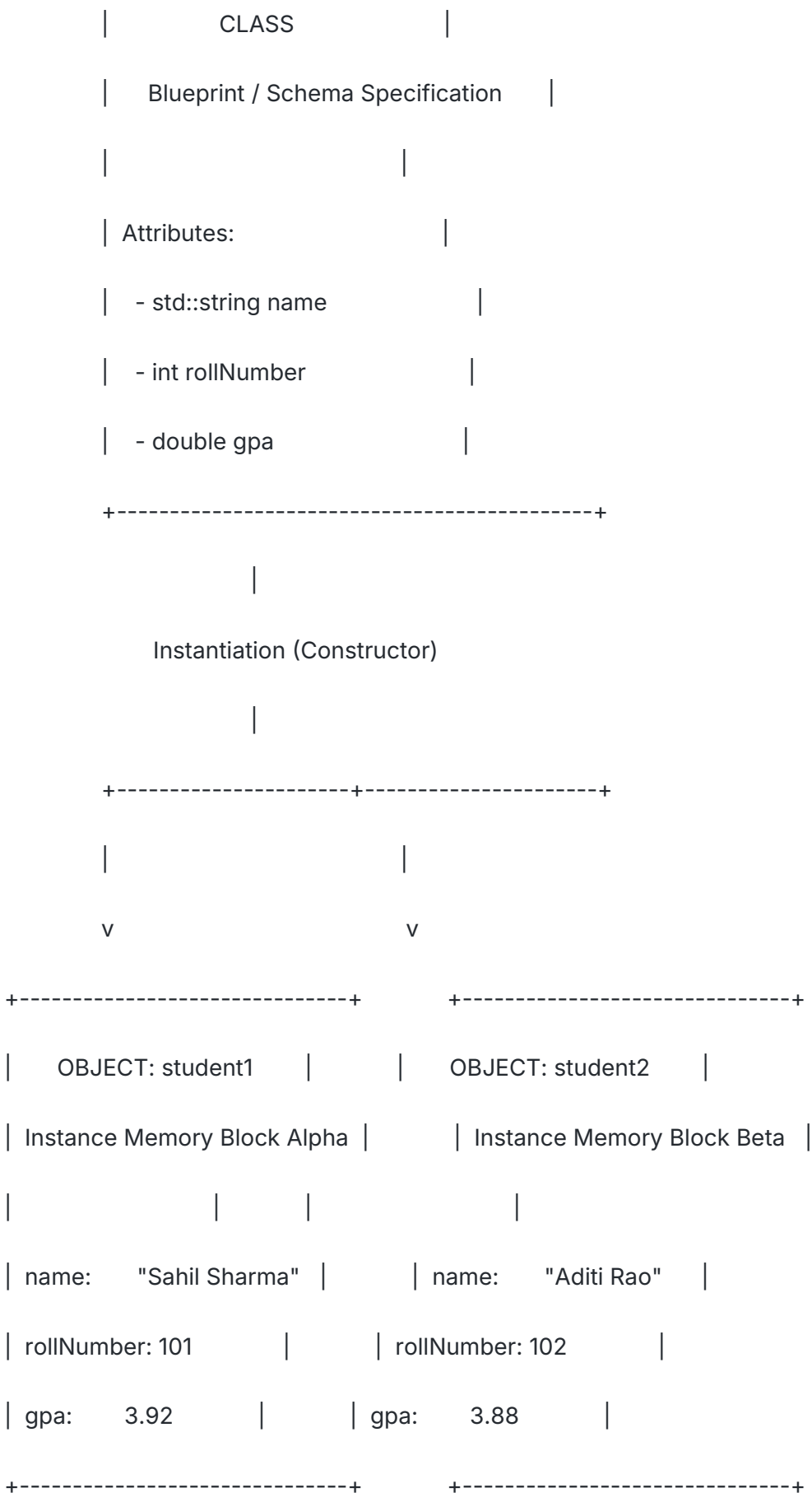
---

1. 🏗️ Core Concepts: Classes and Objects

In procedural programming, maintaining independent variables for related real-world entities (e.g., handling individual strings and integers for student records) becomes disorganized, error-prone, and unsustainable at scale. Object-Oriented Programming (OOP) overcomes this complexity by treating data and operations as integrated systems.

- **Class (The Blueprint)**: A **class** serves as a user-defined template or specification. It outlines the member variables (data attributes) and member functions (behaviors) common to all objects of that category. It specifies what data an entity will have, but does not represent an individual entity itself.
- **Object (The Concrete Manifestation)**: An **object** is an instantiated runtime manifestation of a class. Once instantiated, memory is allocated to hold the object's distinct state. Multiple independent objects can be derived from the exact same class blueprint without colliding.

Conceptual Architecture

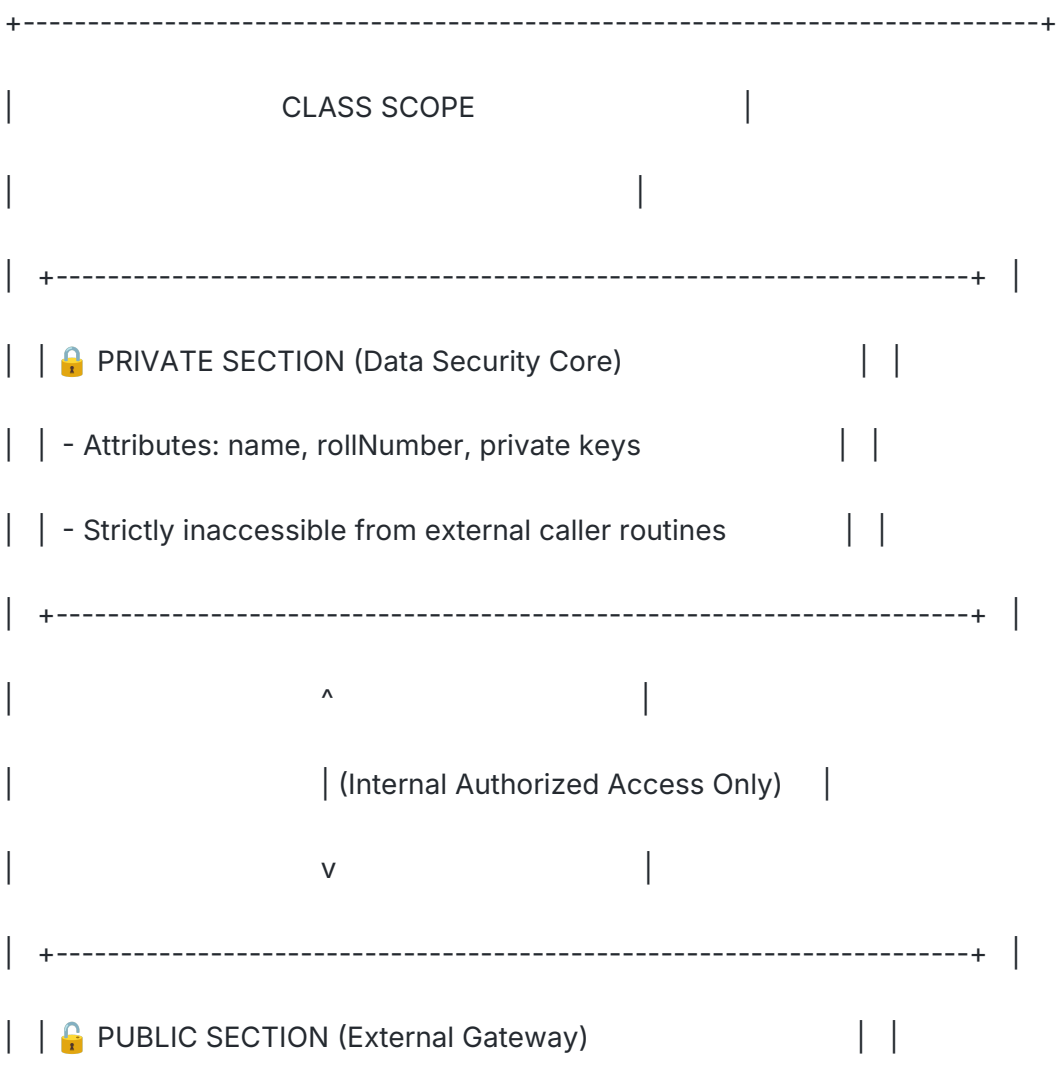

CLASS
Blueprint / Schema Specification
Attributes:
- std::string name
- int rollNumber
- double gpa
Instantiation (Constructor)
OBJECT: student1
Instance Memory Block Alpha
name: "Sahil Sharma"
rollNumber: 101
gpa: 3.92
OBJECT: student2
Instance Memory Block Beta
name: "Aditi Rao"
rollNumber: 102
gpa: 3.88

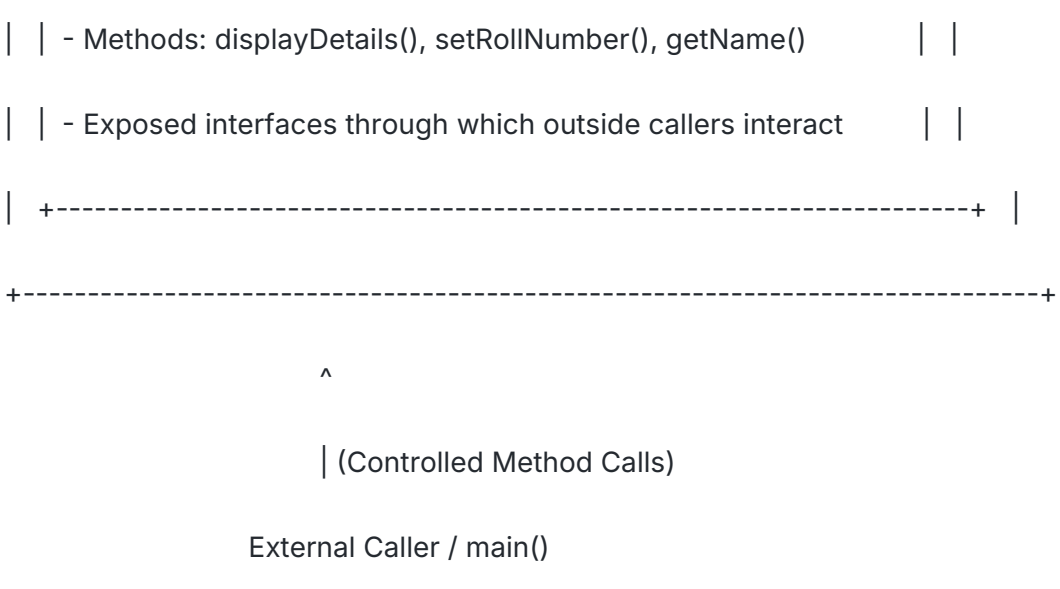
---

2. 🛡️ Why Encapsulation & Data Security

Handling detached attributes across multiple routines leaves programs vulnerable to improper state changes, accidental overwrites, and security lapses.

- **Encapsulation**: Encapsulation is the process of binding data members and the member functions that operate upon them into a cohesive, self-contained unit.
- **Data Hiding & Security**: By restricting direct external access to vital state attributes, a class prevents outside functions from mutating internal states into an inconsistent or invalid condition.
- **Controlled Access**: Access to internal data is managed via well-defined public member functions (getters and setters), ensuring validation rules are uniformly enforced.





---

3. 🔑 Access Modifiers: Public vs. Private vs. Protected

Access modifiers specify the visibility and scope boundaries for class members:

Access Specifier	Accessibility Inside Class	Accessibility in Derived Classes
`private`	Full Access	No Access
`protected`	Full Access	Full Access
`public`	Full Access	Full Access

---

4. ⚖️ Structural Comparison: `struct` vs. `class`

In C++, both `struct` and `class` can contain data members and member functions. The primary distinction lies in their default access boundaries:

Paradigm Feature	Structure (struct)	Class (class)
Default Member Visibility	`public` by default	`private` by default
Default Inheritance Mode	`public` inheritance	`private` inheritance
Primary Architectural Intent	Passive Plain-Old-Data (POD) aggregation	Active domain models with encapsulation and data security
Encapsulation Philosophy	Open access for rapid data	Defensive programming with

Paradigm Feature	Structure (struct)	Class (class)
	grouping	protected internal state

---

5. 🖥️ Implementation: Modern C++ Reference Architecture

Below is a complete C++ implementation illustrating class definition, data hiding, access modifiers, and object instantiation://

```
// ============================================================================
// ============

// File: StudentRecord.cpp

// Topic: Object-Oriented Fundamentals & Encapsulation Reference Architecture

//
// ============================================================================
// ============

#include <iostream>

#include <string>

#include <iomanip>

// --- 1. Class Definition ---

class Student {

private:

    // Private attributes (Secure Data Core - Inaccessible outside class)

    std::string studentName;

    int         rollNumber;

    double      gradePointAverage;

public:

    // Parameterized Constructor
```

```
    Student(const std::string& name, int roll, double gpa)
        : studentName(name), rollNumber(roll), gradePointAverage(gpa) {}

    // Public Getters (Controlled Read Access)
    std::string getName() const {
        return studentName;
    }
    int getRollNumber() const {
        return rollNumber;
    }
    double getGpa() const {
        return gradePointAverage;
    }

    // Public Setter with Input Validation (Defensive Programming)
    void setGpa(double newGpa) {
        if (newGpa >= 0.0 && newGpa <= 4.0) {
            gradePointAverage = newGpa;
        } else {
            std::cout << "[ERROR]: Invalid GPA specified (" << newGpa << "). Mutation
rejected.\n";
        }
    }

    // Public Member Function: Formatted Terminal Display
```

```
    void printStudentProfile() const {
        std::cout << std::left  << std::setw(15) << "Name:"        << studentName         << "\n";
        std::cout << std::left  << std::setw(15) << "Roll Number:" << rollNumber          << "\n";
        std::cout << std::fixed << std::setprecision(2);
        std::cout << std::left  << std::setw(15) << "GPA:"         << gradePointAverage   <<
"\n";
        std::cout << "-----------------------------------------\n";
    }
};

// --- 2. Main Execution ---

int main() {
    std::cout << "=== OOP Core Architecture Demo ===\n\n";

    // Instantiating Concrete Objects from the Student Class Blueprint
    Student studentAlpha("Sahil Sharma", 101, 3.92);
    Student studentBeta("Aditi Rao",    102, 3.88);

    std::cout << "Profile 1 (Object: studentAlpha):\n";
    studentAlpha.printStudentProfile();

    std::cout << "Profile 2 (Object: studentBeta):\n";
    studentBeta.printStudentProfile();

    // Demonstrating Data Security & Encapsulation Enforcement:
    // studentAlpha.rollNumber = 999; // COMPILE ERROR: rollNumber is private

    // Mutating state via authorized public interface
```

```
    std::cout << "Updating studentAlpha GPA via validated public interface...\n";

    studentAlpha.setGpa(3.98);

    studentAlpha.printStudentProfile();

    return 0;

}
```

---

6. 📝 Quick Exam & Interview Checklist

- **Blueprint vs. Concrete Instance**: A class allocates zero execution memory for instance data until an object is instantiated.
- **Primary Purpose of Encapsulation**: Packaging attributes and routines into an atomic entity to shield internal data from unintended side effects.
- **Default Visibility Rule**: Members of a `class` default to `private`; members of a `struct` default to `public`.
- **Principle of Least Privilege**: Default all data members to `private`, exposing only essential interfaces as `public`.
- **Data Security Benefit**: Prevents corruption by external callers and guarantees internal invariant validation through accessors and mutators.

😋 PART -3

💻 C++ Object-Oriented Programming: Classes, Encapsulation & Object Lifecycle

A Comprehensive Guide to Class Blueprints, Data Hiding, Member Invocations, and Memory Management

---

💡 At a Glance: The Architectural Blueprint Analogy

- **Class**: The architectural blueprint or schematic diagram. It defines state variables and operational behavior without allocating concrete application memory.
- **Object**: The constructed building instantiated from the blueprint. It is a concrete entity residing in physical memory with unique state data.
- **Instantiation**: The procedural act of constructing an object instance from the class blueprint in memory.

1. 🏗️ Core Concepts: Classes, Objects & Instantiation

In modern Object-Oriented Programming (OOP) with C++, a **class** serves as a custom, user-defined type that bundles related variables (**attributes / fields / state**) and functions (**behaviors / member functions**) into a single, cohesive unit.

Key Conceptual Distinctions:

1. **The Blueprint (Class)**:
    - Serves as an abstract design template.
    - Defines what attributes an entity will possess and what operations can be executed on those attributes.
    - Does not hold concrete data values itself.
2. **The Concrete Manifestation (Object / Instance)**:
    - When an object variable is declared (e.g., `Student s1;`), the compiler allocates memory cells to hold individual state values.
    - Each object (`s1`, `s2`) maintains its own independent storage space for its attributes (`rollNumber`, `name`, `gpa`, `age`).
3. **Class Instantiation**:
    - The formal terminology for creating an object of a class is **Class Instantiation**.

- An object is formally defined as an **instance of a class**.

2. 🛡️ Why Encapsulation & Data Hiding

A fundamental pillar of robust software engineering is **Data Hiding** (Encapsulation). Exposing internal object state directly to external routines leads to brittle, bug-prone architectures.

The Problem of Direct Member Access:

In C++, if members are placed in the private scope, attempting direct assignment or retrieval from outside the class (such as inside `main()`):// ILLEGAL & REJECTED BY COMPILER

s1.name = "Sahil";

std::cout << s1.rollNumber;

This triggers a strict compiler error: error: 'name' is a private member of 'Student'.

The Solution: Public Interface Contract

To safeguard object integrity, internal fields are marked private, while a controlled set of member functions is exposed under **`public`**:

- **Setters / Mutators (`setData`)**: Validate and receive external inputs into the object's private members.
- **Getters / Accessors (`getData`)**: Format and present internal member values to external consumers without compromising direct write access.

3. 🔑 Access Modifiers & Visibility Rules

Access specifiers determine the visibility scope of members within and outside class boundaries.

Access Specifier	Accessibility Inside Class	Accessibility from External Functions (main)
`private`	Full Access	**Strictly Forbidden (Illegal)**
`public`	Full Access	**Direct Access Permitted**

Access Specifier	Accessibility Inside Class	Accessibility from External Functions (main)
`protected`	Full Access	**Strictly Forbidden**

⚠ Key Takeaway on Access Boundaries

Attempting to read or write a private member directly outside class scope violates encapsulation principles and will not compile. Member functions defined within the class act as authorized intermediaries.

4. ⚙ Member Functions & Function Invocation

Functions declared inside a C++ class are explicitly referred to as **Member Functions** (contrasted with "methods" in Java parlance).

1. Defining Behavior

While variables represent state and attributes, member functions define **behavior**:

- `setData()`: Fully responsible for taking user input via standard input streams (`std::cin`) and assigning values directly to the invoking object's private attributes.
- `getData()`: Responsible for displaying member attributes clearly to the console output stream (`std::cout`).

2. Function Invocation

Invoking a member function on a concrete instance requires the dot operator (`.`):s1.setData(); // Function Invocation for instance s1

s1.getData(); // Dispatches display behavior on instance s1

When `s1.setData()` is invoked, the function operates strictly within the context and memory block of s1. Subsequent calls on `s2.setData()` operate independently on `s2`.

5. 🔄 Object Lifecycle: Constructors, Destructors & Memory Management

Every C++ object goes through a deterministic lifecycle in memory:[Declaration / Instantiation] ⟶ [Constructor Invocation] ⟶ [Memory Allocation & Initialization]



1. Allocation & Construction

- When an object is instantiated, the system allocates physical memory for its members.
- The **Constructor** automatically initializes member variables to valid default states.

2. Teardown & Deallocation

- When an object finishes its task and leaves scope (e.g., reaching the end of `main()`), its allocated memory must be freed.
- In C++, the **Destructor** (`~Student()`) is executed automatically, releasing resources and deallocating memory.

6. 🖥️ Implementation: Modern Developer Code (`Student.cpp`)

Below is the complete, self-contained implementation written in a clean, modern VS Code editorial standard://

```
================================================================
============
```

```
// File: Student.cpp

// Topic: C++ Object-Oriented Programming - Class Blueprint & Member Functions

//
================================================================
============
```

```
#include <iostream>
```

```
#include <string>

// --- 1. Class Definition ---

class Student {

private:

    // Private attributes (State / Data Hiding)

    int         rollNumber;

    std::string name;

    float       gpa;

    int         age;

public:

    // --- 1.1 Lifecycle: Constructor ---

    // Automatically executes upon object instantiation in memory

    Student() {

        rollNumber = 0;

        name       = "";

        gpa        = 0.0f;

        age        = 0;

    }

    // --- 1.2 Behavior: Mutator (Set Data) ---

    // Public member function responsible for taking inputs into private members

    void setData() {

        std::cout << "Enter Roll Number: ";
```

```
    std::cin  >> rollNumber;
    std::cout << "Enter Student Name: ";
    std::cin  >> name;
    std::cout << "Enter Grade Points (GPA): ";
    std::cin  >> gpa;
    std::cout << "Enter Age: ";
    std::cin  >> age;
}
// --- 1.3 Behavior: Accessor (Get Data) ---
// Public member function responsible for formatting and displaying details
void getData() const {
    std::cout << "\n----------------------------------------\n";
    std::cout << "          STUDENT RECORD PROFILE        \n";
    std::cout << "----------------------------------------\n";
    std::cout << "Roll Number : " << rollNumber << "\n";
    std::cout << "Name        : " << name       << "\n";
    std::cout << "GPA         : " << gpa        << "\n";
    std::cout << "Age         : " << age        << "\n";
    std::cout << "----------------------------------------\n";
}
// --- 1.4 Lifecycle: Destructor ---
```

```
    // Automatically executes when the object goes out of scope to release memory
    ~Student() {
        // Resource cleanup and memory deallocation logic
    }
};

// --- 2. Main Execution Routine ---
int main() {
    // 2.1 Object Instantiation (Creating concrete instances from the blueprint)
    Student s1;
    Student s2;
    // 2.2 Function Invocation: Taking inputs for Student 1
    std::cout << "=========================================\n";
    std::cout << "       RECORD ENTRY: STUDENT 1         \n";
    std::cout << "=========================================\n";
    s1.setData();
    // 2.3 Function Invocation: Taking inputs for Student 2
    std::cout << "\n=========================================\n";
    std::cout << "       RECORD ENTRY: STUDENT 2         \n";
    std::cout << "=========================================\n";
    s2.setData();
    // 2.4 Function Invocation: Displaying populated student records
    std::cout << "\n=========================================\n";
```

```
    std::cout << "      DISPLAYING GENERATED RECORDS     \n";
    std::cout << "=========================================\n";
    s1.getData();
    s2.getData();
    return 0;
}
```

7. 📊 Comprehensive Reference & Terminology Comparison

Dimension	C++ Paradigm	Java Equivalent
Blueprint	`class`	`class`
Instance	`object` (`s1`, `s2`)	`object reference`
State	Members / Attributes / Fields	Fields / Instance Variables
Behavior	**Member Functions**	**Member Methods**
Setup Routine	`Constructor` (`Student()`)	`Constructor` (`Student()`)
Teardown Routine	**Destructor** (`~Student()`)	Garbage Collector (Implicit)
Call Mechanism	Function Invocation (`.`)	Method Invocation (`.`)

8. 📌 Exam Summary & Quick Checklist

📝 Quick Exam Checklist

- **Classes vs Objects**: Remember that a class consumes zero object instance memory on definition; memory allocation occurs solely upon object instantiation.
- **Data Hiding**: Direct access to private members outside class scope (e.g., `s1.name` inside `main`) causes a compilation error.
- **Encapsulation Architecture**: Maintain private attributes and provide public member functions (`setData` / `getData`) as safe access interfaces.
- **Function Invocation**: Member functions are invoked via the dot operator (`instance.function()`) and operate on the calling instance's specific memory space.

Lifecycle Mechanics: Constructors initialize memory when objects are created; destructors deallocate memory when objects expire.

😌 PART -4

💻 Object-Oriented Programming: Memory Lifecycle & Friend Functions , Polymorphism

Executive Developer notes • Core C++ : Encapsulation, Object Lifecycles, and Controlled Access

---

💡 Key Mental Models & Architectural Intuition

• Encapsulation & Private Data: Think of a private apartment. Outsiders cannot walk in or touch your
permission.
• Friend Function: A trusted friend whom you give a duplicate key. They are not a member of the hou
class), but they have special clearance to access your private possessions.
• Constructor: The builder and interior designer who sets up the foundation and allocates resources
instantiated.
• Destructor: The cleanup crew that deallocates memory and reclaims resources before the apartme

1. 🏗️ Core Architectural Concepts

🛡️ Why Encapsulation Matters

In C++, data security and modular integrity are enforced through **access specifiers**. By default, all members of a `class` are **private**. This prevents arbitrary, unsafe mutation from external code. However, strictly closing off internals can occasionally hinder operations like operator overloading, inter-class communication, and specialized logging.

🔑 The Friend Function Paradigm

- **Definition:** A **friend function** is a non-member function defined outside the class scope that is granted full access rights to the class's **private** and **protected** members.
- **Declaration:** Declared inside the class body preceded by the keyword `friend`.
- **Definition:** Defined outside the class without the `friend` keyword and without any class scope resolution operator (`ClassName::`).
- **Non-Member Status:** It is **not** a member function; therefore, it cannot be invoked via the `.` (dot) or `->` (arrow) operator on an object. Instead, objects are passed as arguments to it.

2. ⚙️ Memory Management: Constructors & Destructors

Every object instantiated in C++ claims a specific footprint in memory. Proper resource stewardship requires deterministic setup and teardown:

- **Constructor Lifecycle:** When an object is instantiated, the constructor automatically executes, allocating memory buffers and initializing state variables.
- **Operational Scope:** The object resides in memory (on the stack or heap) while executing business logic.
- **Destructor Teardown:** When an object leaves its local scope or is explicitly deleted, its **destructor** is called. This frees system buffers, closes file handles, and prevents critical **memory leaks**.

> constructor is a special member function which has the same name as that of the class.

Student s1 :

- s1 is not a variable
- Constructors will be invoked at the time of class instantiation .
- We can give initial values of (constructor) in public values .
- This is a collection of variables .

Roll no
Name
Gpa
Age

- A default constructor is also known as a parameterised constructor .

3. 📊 Technical Comparison & Reference Tables

Access Modifiers & Visibility Matrix

Specifier	Accessible Inside Class?	Accessible by Derived Classes?	Accessible by Ou Functions?
`private`	✅ Yes	❌ No	❌ No
`protected`	✅ Yes	✅ Yes	❌ No
`public`	✅ Yes	✅ Yes	✅ Yes

Constructor vs. Destructor Lifecycle Matrix

Dimension	Constructor	Destructor
Naming Syntax	`ClassName()`	`~ClassName()` (Prefixed with tilde ~)
Invocation	Automatically called on object creation	Automatically called upon destruction / out-of-scope
Primary Role	Memory allocation & state initialization	Memory deallocation & resource cleanup
Return Type	No return type (not even `void`)	No return type (not even `void`)
Overloading	Supports multiple overloaded constructors	Cannot be overloaded; exactly one per class
Parameters	Accepts default, parameterized, or copy arguments	Never accepts any parameters

4. 🖥️ Implementation: Modern C++ Blueprint

```
// ================================================================================
// File: memory_and_friends.cpp
// Topic: Friend Functions & Resource Management in C++
```

```cpp
//
================================================================================
========

#include <iostream>
#include <string>

// --- 1. Class Definition ---
class SecureVault {
private:
    int         secretKey;
    double*     dataBuffer;

public:
    // Parameterized Constructor
    SecureVault(int key, double initialValue) {
        secretKey   = key;
        dataBuffer  = new double(initialValue); // Dynamic memory
allocation
        std::cout << "[CONSTRUCTOR] Vault initialized with Key: " <<
secretKey << std::endl;
    }

    // Friend Function Declaration
    friend void inspectVault(const SecureVault& vault);

    // Destructor for Memory Deallocation
    ~SecureVault() {
        delete dataBuffer; // Free allocated memory to prevent
memory leaks
        std::cout << "[DESTRUCTOR]  Memory deallocated for Vault
with Key: " << secretKey << std::endl;
    }
};

// --- 2. Friend Function Implementation (Outside Class Scope) ---
void inspectVault(const SecureVault& vault) {
    // Directly accesses private members without getter methods
```

```
    std::cout << "\n[FRIEND ACCESS] Reading private secretKey: " <<
vault.secretKey << std::endl;
    std::cout << "[FRIEND ACCESS] Reading private dataBuffer: " <<
*(vault.dataBuffer) << "\n" << std::endl;
}

// --- 3. Main Execution ---
int main() {
    std::cout << "--- Creating Object in Local Scope ---" <<
std::endl;
    {
        SecureVault myVault(4201, 98.75);
        inspectVault(myVault);
    } // Object goes out of scope here -> Destructor triggers
automatically

    std::cout << "--- Execution Completed ---" << std::endl;
    return 0;
}
```

ठ

---

5. 📝 Quick Exam & Interview Checklist

-

📌 Key Verification Points

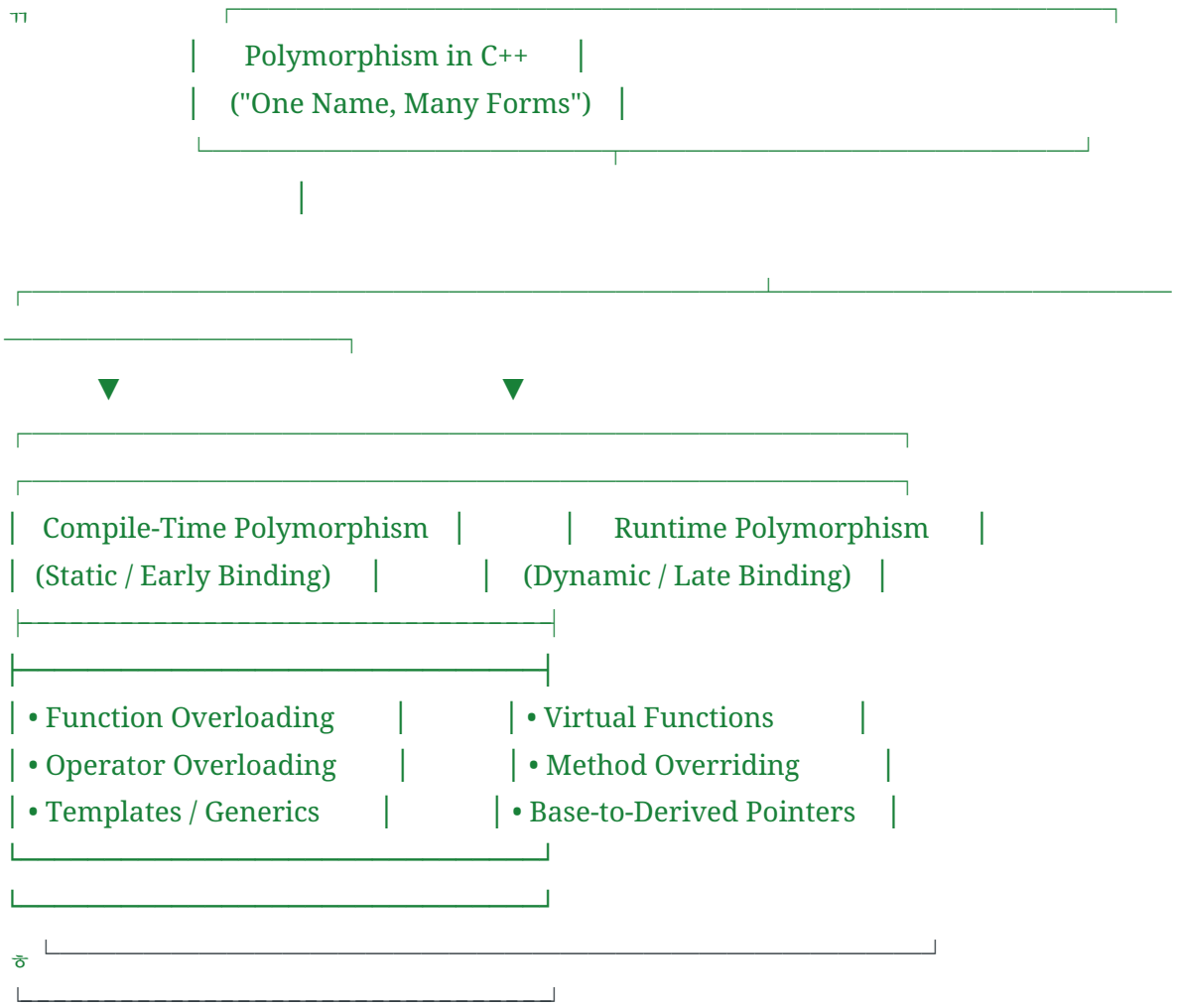
• Keyword Rules: The friend keyword is written only in the class declaration, never in the external fu
• No this Pointer: Because a friend function is not a member function, it does not possess a hidden t
objects as explicit arguments.
• Friendship is Non-Reciprocal: If Class A is a friend of Class B, Class B is not automatically a friend
granted.
• Friendship is Not Inherited: Derived classes do not inherit friend access from base classes.
• Destructor Signature: Must have the same name as the class preceded by ~, takes zero argument
• RAII Paradigm: Resource Acquisition Is Initialization binds resource lifecycle directly to object lifeti

💻 Polymorphism & Method Overloading

An Executive Technical Guide to Compile-Time Binding, Function Signatures, and Operator Overloading💡 **At a *Glance: The Universal Interface Principle***
Polymorphism stems from the Greek words poly (many) and morph (form)—literally meaning "one name, many forms." In software architecture, it allows a single interface or identifier (such as add()) to perform different operations depending on the context, parameter types, or invocation targets.

🏗️ 1. Core Concepts: The Polymorphism Architecture

Polymorphism is one of the four foundational pillars of Object-Oriented Programming (OOP). It decouples what an operation is called from **how** it is executed.



The Two Fundamental Disciplines:

1. **Compile-Time Polymorphism**: The binding between the function call and the underlying function body is finalized by the compiler during compilation.
2. **Runtime Polymorphism**: The function call resolution is deferred until runtime, typically resolved via **virtual method tables (vtables)** and base-class pointers or references.

🛡️ 2. Taxonomy & Comparison: Static vs. Dynamic Binding

Understanding the alternate terminology is critical for system design reviews and technical examinations.

Reference Table: Binding Mechanisms & Synonyms

Dimensions	Compile-Time Polymorphism	Runtime Polymorphism
Primary Synonyms	**Static Polymorphism**, **Early Binding**, **Static Binding**	**Dynamic Polymorphism**, **Late Binding**, **Dynamic Binding**
Core Mechanisms	**Function Overloading**, **Operator Overloading**	**Method Overriding** via `virtual` functions
Resolution Phase	Compiler stage (`g++` / `clang` translation phase)	Execution stage (runtime memory evaluation)
Dispatch Mechanism	Name mangling & static symbol resolution	**VTable** (Virtual Method Table) & **VPtr** indirection
Entity Target	Direct objects, static references, value types	Base class pointers (`Base*`) & references (`Base&`)
Performance Profile	**Zero runtime overhead**; supports compiler inlining	Minor indirection cost due to pointer dereferencing
Flexibility	High type-safety, fixed at compile time	High extensibility, extensible via plugin hierarchies

🔑 3. Mechanics of Function / Method Overloading

Function Overloading occurs when multiple member functions within the same scope share the **identical identifier**, but possess distinct **parameter signatures**.

```
Function Signature = Function Name + Parameter List (Type, Count, Order)
```

The Overloading Differentiation Rules

Overloading is permitted **if and only if** the parameter signatures differ by:

1. **Parameter Count**: Changing the number of arguments (e.g., 2 integers vs. 3 integers).
2. **Parameter Data Types**: Supplying distinct primitive or user-defined types (e.g., `(int, int)` vs. `(int, double)`).
3. **Parameter Ordering**: Alternating type placement (e.g., `(int, double)` vs. `(double, int)`).

⚠️ **Compiler Directive: Return Type Invariance**Altering **only** the return type (e.g., int add() vs. double add()) is not valid overloading. The compiler cannot determine which function to invoke when the return value is discarded or implicitly converted, resulting in a compile-time ambiguity error.

⚙️ 4. Compiler Resolution: Under the Hood

How does the compiler distinguish between multiple functions sharing the same name?

1. **Invocation Inspection**: When a function call like `s1.add(10, 20)` is parsed, the compiler checks the actual arguments passed (`int`, `int`).
2. **Candidate Matching**: The compiler scans the class's symbol table for an exact match among all visible prototypes.
3. **Name Mangling (Decorated Names)**: The compiler encodes function signatures into unique internal symbol names (e.g., `_ZN6Sample3addEii` for `add(int, int)` and `_ZN6Sample3addEid` for `add(int, double)`).
4. **Early Binding**: The mangled symbol's direct memory address is written directly into the binary assembly call site (e.g., `call _ZN6Sample3addEii`).

🖥️ 5. Implementation: The `Sample` Class Engine

Enterprise-grade C++ implementation demonstrating member function overloading:

```
דד//
================================================================================
// File:    FunctionOverloadingSample.cpp
// Topic:   Compile-Time Polymorphism via Member Function Overloading
// Standard: C++17 / C++20 Compatible
//
================================================================================

#include <iostream>
#include <iomanip>

// --- 1. Class Definition ---
class Sample {
public:
    // Overload 1: Two Integer Parameters
    int add(int x, int y) {
        return x + y;
    }

    // Overload 2: Mixed Parameters (Integer + Double)
    double add(int x, double y) {
```

```
        return x + y;
    }

    // Overload 3: Three Integer Parameters
    int add(int x, int y, int z) {
        return x + y + z;
    }

    // Overload 4: Zero Parameters (Default / Procedural Variant)
    void add() {
        std::cout << "[LOG] Executing non-parameterized add(): No
values to compute.\n";
    }
};

// --- 2. Main Execution ---
int main() {
    // Instantiate static instance
    Sample s1;
    std::cout << "================================================\n";
    std::cout << "      C++ FUNCTION OVERLOADING DEMONSTRATION   \n";
    std::cout << "================================================\n\n";

    // Call 1: Matches add(int, int) -> Resolved at Compile Time
    int sumTwoInts = s1.add(10, 20);
    std::cout << "Addition of two integers (10, 20)        : "
              << sumTwoInts << "\n";

    // Call 2: Matches add(int, double) -> Resolved at Compile Time
    double sumMixed = s1.add(10, 11.2849);
    std::cout << "Addition of int & double (10, 11.2849)   : "
              << std::fixed << std::setprecision(4) << sumMixed <<
"\n";

    // Call 3: Matches add(int, int, int) -> Resolved at Compile Time
    int sumThreeInts = s1.add(10, 20, 35);
    std::cout << "Addition of three integers (10, 20, 35)  : "
              << sumThreeInts << "\n";

    // Call 4: Matches add() -> Resolved at Compile Time
```

```
    std::cout << "Addition with no arguments                : ";
    s1.add();
    std::cout << "\n================================================\n";

    return 0;
}
```

🔬 Execution Output & Mapping Analysis

Call Site	Actual Arguments	Target Function Prototype
`s1.add(10, 20)`	`(int, int)`	`int add(int, int)`
`s1.add(10, 11.2849)`	`(int, double)`	`double add(int, double)`
`s1.add(10, 20, 35)`	`(int, int, int)`	`int add(int, int, int)`
`s1.add()`	`()`	`void add()`

🔢 6. Operator Overloading: Custom Type Arithmetic

Operator Overloading extends compile-time polymorphism to native C++ symbols (+, -, *, /). Rather than restricting operators to primitive numeric types, developers can define custom operational behavior for user-defined types (e.g., Complex, Matrix, Vector).

Mathematical Framework: Complex Arithmetic

A complex number is defined as Z = a + ib, where:

- ***a = Real Part***
- ***b = Imaginary Part***
- ***i = sqrt(-1)***

Given C1 = a1 + ib1 and C2 = a2 + ib2:

- ***Addition: C_sum = (a1 + a2) + i(b1 + b2)***
- ***Subtraction: C_diff = (a1 - a2) + i(b1 - b2)***
- ***Multiplication: C_prod = (a1a2 – b1b2) + i(a1b2 + b1a2)***
- ***Division: C_quot = ((a1a2 + b1b2) + i(b1a2 - a1b2)) / (a2^2 + b2^2)***

Implementation: The Enterprise Complex Class

```
ד//
================================================================================
// File:    ComplexArithmetic.cpp
// Topic:   Operator & Method Overloading for Complex Number Mathematics
//
================================================================================

#include <iostream>

class Complex {
private:
    int real;
    int imag;

public:
    // --- Lifecycle Constructors ---
    Complex() : real(0), imag(0) {}
    Complex(int r, int i) : real(r), imag(i) {}

    // --- I/O Member Functions ---
    void read() {
        std::cout << "Enter Real Part      : ";
        std::cin  >> real;
        std::cout << "Enter Imaginary Part : ";
        std::cin  >> imag;
    }

    void display() const {
        if (imag == 0) {
            std::cout << real;
        } else if (imag > 0) {
            std::cout << real << " + " << imag << "i";
        } else {
            std::cout << real << " - " << (-imag) << "i";
        }
```

```
        std::cout << "\n";
    }

    // --- Functional Member Methods ---
    Complex add(const Complex& c2) const {
        return Complex(real + c2.real, imag + c2.imag);
    }

    Complex subtract(const Complex& c2) const {
        return Complex(real - c2.real, imag - c2.imag);
    }

    // --- Operator Overload: Compile-Time '+' Binding ---
    Complex operator+(const Complex& c2) const {
        return Complex(real + c2.real, imag + c2.imag);
    }
};

// --- Execution Testing ---
int main() {
    Complex c1(2, 3);   // 2 + 3i
    Complex c2(4, 2);   // 4 + 2i

    // Method Call Syntax
    Complex cAddMethod = c1.add(c2);

    // Operator Overloaded Syntax: Resolved at compile time to
c1.operator+(c2)
    Complex cAddOp = c1 + c2;

    // Display Results
    std::cout << "C1              : "; c1.display();
    std::cout << "C2              : "; c2.display();
    std::cout << "Sum (Method)    : "; cAddMethod.display();
    std::cout << "Sum (Operator +): "; cAddOp.display();

    return 0;
}
```

🗒️📌 7. Exam Summary & High-Yield Takeaways

📝 **Quick Exam Checklist**

- **Definition**: Polymorphism = *"One name, many forms."*
- **Taxonomy**:
    - **Compile-Time**: Function Overloading, Operator Overloading, Early Binding, Static Binding.
    - **Runtime**: Function Overriding, Virtual Functions, Late Binding, Dynamic Binding.
- **Overloading Rule**: Differentiate only by parameter **count**, **type**, or **order**. Return type differences alone are **illegal**.
- **Compiler Mechanism**: Function calls are linked directly to definitions via **name mangling** during compilation.
- **Operator Overloading**: Extends predefined operator symbols (`+`, `-`) to custom class instances without changing operator precedence or arity.

Extended Operator Overloading & Friend Functions

6.1 Friend Function Architecture for Binary Operations

- **Encapsulation vs. Friend Functions**: Friend functions provide controlled access to private members for external non-member functions without breaking encapsulation principles.
- **Syntax & Rules**: Declared with the `friend` keyword inside the class definition; defined outside without the scope resolution operator (`::`) and without receiving an implicit `this` pointer.
- **Multiplication Friend Function**: Formula signature: `friend Complex mul(const Complex& c1, const Complex& c2);` where (a + ib)(c + id) = (ac - bd) + i(ad + bc).

6.2 Binary Operator Overloading Calling Conventions: Implicit vs. Explicit Operands

- The expression `c1 / c2` implicitly translates to the member function called `c1.operator/(c2)`.

- The left operand (`c1`) is passed implicitly via this pointer, while the right operand (`c2`) is passed explicitly as an argument.
- Overloading division (`/`) is necessary because native C++ division operators only support primitive data types, not user-defined complex structures.

6.3 Mathematical Derivations of Complex Division

- **Conjugate Rationalization**: (a + ib) / (c + id) = ((ac + bd) + i(bc - ad)) / (c^2 + d^2)
- **Real Part Formula**: (real * c2.real + imag * c2.imag) / (c2.real^2 + c2.imag^2)
- **Imaginary Part Formula**: (imag * c2.real - real * c2.imag) / (c2.real^2 + c2.imag^2)

6.4 Complete Extended Complex Class C++ Implementation

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real;
    double image;
public:
    Complex(double r = 0.0, double i = 0.0) : real(r), imag(i)
{}

    void read() {
        cout << "Enter real and imaginary parts: ";
        cin >> real >> imag;
    }

    void display() const {
        cout << real << (imag >= 0 ? " + " : " - ") <<
abs(imag) << "i" << endl;
    }
```

```
    Complex operator+(const Complex& c2) const {
        return Complex(real + c2.real, imag + c2.imag);
    }

    Complex operator-(const Complex& c2) const {
        return Complex(real - c2.real, imag - c2.imag);
    }

    Complex operator/(const Complex& c2) const {
        double denom = c2.real * c2.real + c2.imag * c2.imag;
        double r = (real * c2.real + imag * c2.imag) / denom;
        double i = (imag * c2.real - real * c2.imag) / denom;
        return Complex(r, i);
    }

    friend Complex mul(const Complex& c1, const Complex& c2);
};

Complex mul(const Complex& c1, const Complex& c2) {
    double r = c1.real * c2.real - c1.imag * c2.imag;
    double i = c1.real * c2.imag + c1.imag * c2.real;
    return Complex(r, i);
}

int main() {
    Complex c1(2.0, 3.0);
    Complex c2(1.0, 2.0);

    cout << "c1 = "; c1.display();
    cout << "c2 = "; c2.display();

    Complex sum = c1 + c2;
    Complex diff = c1 - c2;
    Complex prod = mul(c1, c2);
```

```
    Complex quot = c1 / c2;

    cout << "Sum: "; sum.display();
    cout << "Difference: "; diff.display();
    cout << "Product (friend mul): "; prod.display();
    cout << "Quotient: "; quot.display();

    return 0;
}
```

ㅎ

Polymorphism & Memory Architecture

Dynamic Binding, Vtable Internals, and Object Lifecycle

💡 Core Intuition & Executive Summary

- **Binding & Pointers:** Polymorphism allows entities to exhibit multiple behaviors. Compile-time polymorphism resolves calls during compilation, while runtime polymorphism (late binding) delays resolution to execution. Non-virtual binding defaults strictly to the pointer's declared type rather than the underlying instance.
- **Virtual Dispatch & Lifecycle:** Adding the `virtual` keyword enables dynamic dispatch via a hidden pointer (`vptr`) pointing to a class-specific address lookup table (`vtable`). Polymorphic base classes require virtual destructors to prevent memory leaks, whereas constructors can never be virtual.

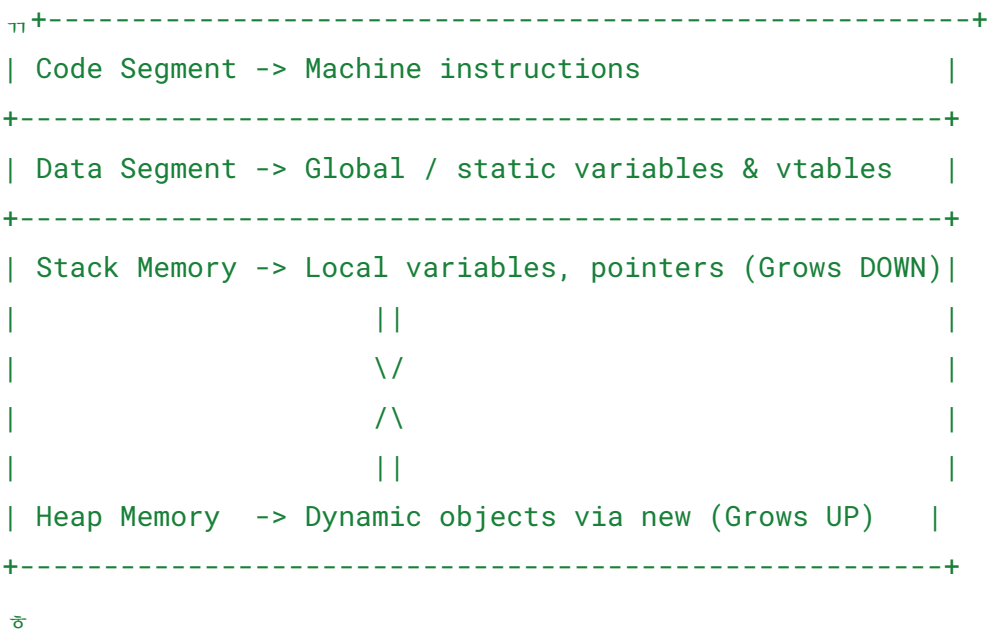
1. Core Concepts & Terminology

- **Runtime Polymorphism:** Function resolution occurs dynamically at execution time (late/dynamic binding).
- **Function Overriding:** A derived class redefines an inherited base class member function using the exact same signature.
- **Function Signature:** Consists of the function name and parameter type list.
- **Function Overloading vs. Overriding:**
  - **Overloading:** Multiple functions in the same class scope sharing a name but having distinct parameters.
  - **Overriding:** Identical function signatures redefined across derived class boundaries.

2. Comparative Architecture: Overloading vs. Overriding

Feature / Metric	Function Overloading	Function Overriding
Binding Type	Early / Static (Compile-time)	Late / Dynamic (Runtime)
Scope	Within the same class	Across base and derived classes
Signature	Different parameters required	Must have exact same signature
Resolution	Resolved during compilation	Resolved at runtime via virtual dispatch

3. Runtime Memory Layout & Allocation



Static vs. Dynamic Allocation

1. **Static Allocation (Stack / Data Segment):** Variables receive named memory addresses assigned by the compiler at compile-time.
2. **Dynamic Allocation (Heap via `new`):**
    - Allocated strictly at execution time without dedicated variable names.
    - Heap objects exist as raw memory addresses and must be accessed via pointers.
    - Dereferencing member functions requires the arrow operator (`->`) instead of standard dot notation.

4. Member Dispatch Mechanics (Non-Virtual)

When invoking member functions through pointers without the `virtual` keyword:

1. **Base Pointer to Base Object:**
    - Invokes Base method directly (`bp1->fun()` → `Base::fun`).
2. **Derived Pointer to Derived Object:**
    - Invokes the Derived method directly (`dp1->fun()` → `Derived::fun`).
3. **Base Pointer to Derived Object (The Polymorphic Dilemma):**
    - Assigning `Base* bp2 = new Derived();` is valid in C++.
    - Without `virtual`, calls default strictly to the static pointer type (`Base*`), executing `Base::fun` instead of the derived implementation.

5. Non-Virtual Binding Example

```
#include <iostream>

class Base {
public:
    void fun() { std::cout << "Base fun" << std::endl; }
};

class Derived : public Base {
public:
    void fun() { std::cout << "Derived fun" << std::endl; }
};

int main() {
```

```
    Base* bp1 = new Base();
    bp1->fun(); // Output: Base fun

    Derived* dp1 = new Derived();
    dp1->fun(); // Output: Derived fun

    Base* bp2 = new Derived();
    bp2->fun(); // Output: Base fun (Static binding!)

    delete bp1; delete dp1; delete bp2;
    return 0;
}
```

🛠 6. Under the Hood: Vtable & Vptr Internals

When a class defines or inherits at least one **`virtual` function**, the compiler automatically inserts two hidden mechanisms to resolve function calls at runtime:

- **Virtual Table (vtable):** A static array of function pointers constructed at compile-time per polymorphic class, storing addresses of overridden virtual methods.
- **Virtual Pointer (vptr):** A hidden pointer injected into each object instance by the compiler. Initialized during construction to reference the corresponding class vtable.

Virtual Dispatch Mechanism

```
┌─[ Stack Pointer: Base* bp ]
        |
        v (Points to Heap Instance)
[ Heap Instance: Derived Object ]
    ├── Member: *__vptr ──> [ Read-Only Data: Derived vtable ]
    └── Data: members           ├── Slot 1: &Derived::fun
                                └── Slot 2: &Base::otherFunc
```

🛠 Vtable vs. Vptr Comparison

Feature	Virtual Table (vtable)	Virtual Pointer (vptr)
Scope	One per class (shared across	One per object instance

Feature	Virtual Table (vtable)	Virtual Pointer (vptr)
	instances)	
Creation	Compile-time (stored in read-only data)	Runtime (allocated inside object memory)
Object Overhead	Zero direct instance memory cost	Increases size by 1 pointer (4 or 8 bytes)

7. Virtual Destructors & Lifecycle Mechanics

When managing dynamic objects polymorphically (`Base* bp = new Derived();`), deleting the pointer (`delete bp;`) triggers destructor execution.

- **Non-Virtual Destructor Hazard:** Deleting a derived instance through a non-virtual base pointer skips `~Derived()` entirely, leaking derived heap resources.
- **Virtual Destructor Solution:** Declaring `virtual ~Base()` routes destruction through the vtable, ensuring `~Derived()` executes first before chaining up to `~Base()`.

8. Architectural Constraints: Why No Virtual Constructors

C++ strictly forbids declaring a constructor as `virtual` (`virtual Base() {}` throws a compiler error) for two fundamental architectural reasons:

1. **The Vptr Initialization Paradox:** Virtual dispatch relies on a valid `vptr`. Since constructors set up the `vptr`, virtual lookup is impossible before constructor execution finishes.
    - Virtual calls require an established vptr address lookup in memory.
    - Constructors build the object and initialize its vptr; virtual dispatch cannot occur prior.
2. **Concrete Type Requirements:** Memory allocation requires knowing the exact size (`sizeof`) of the dynamic type at compile-time.
    - Dynamic object allocation needs deterministic instance sizing before runtime creation.

9. Polymorphic Virtual Destructor Implementation

```
#include <iostream>

class Base {
```

```
public:
    Base() { std::cout << "[Constructor] Base initialized" <<
std::endl; }
    virtual void fun() { std::cout << "[Dispatch]    Base::fun()" <<
std::endl; }
    virtual ~Base() { std::cout << "[Destructor]  Base cleaned up" <<
std::endl; }
};

class Derived : public Base {
private:
    int* heapResource;
public:
    Derived() {
        heapResource = new int(42);
        std::cout << "[Constructor] Derived initialized" << std::endl;
    }
    void fun() override { std::cout << "[Dispatch]    Derived::fun()
via vptr -> vtable" << std::endl; }
    ~Derived() override {
        delete heapResource;
        std::cout << "[Destructor]  Derived cleaned up" << std::endl;
    }
};

int main() {
    Base* bp = new Derived();
    bp->fun();   // Dynamic dispatch to Derived::fun()
    delete bp;   // Triggers ~Derived() then ~Base()
    return 0;
}
```

🚦📌 10. Exam & Interview Checklist

- **Runtime Polymorphism:** Execution binding resolved dynamically at runtime (late binding / dynamic dispatch).
- **Signature Rule:** Overriding requires matching function name and parameter types.

- **Memory Segments:** Code (instructions), Data (globals/vtables), Stack (local pointers), Heap (dynamic objects).
- **Static Type Priority:** Non-virtual calls resolve strictly via declared pointer type.
- **Virtual Cost:** First virtual function adds 1 pointer size (vptr) per object instance.

Destructor Rule: Polymorphic base classes must define a virtual destructor.

METHOD OVERLOADING ⟶

```
#include <iostream>
using namespace std;
class Complex{
   private: int a;
            int b;
   public:
          void readComplexNumber(){
             cout<<"\nEnter real part: ";
             cin>>a;
             cout<<"\nEnter imaginary part: ";
             cin>>b;
          }
          void displayComplexNumber(){
             if(b == 0){
                cout<<a<<"+i0";
             }
             else if(b == 1){
                cout<<a<<" + i";
             }
             else if(b == -1){
                cout<<a<<" - i"<<(-1)*b;
             }
```

```
            else if(b < 0){
                cout<<a<<" - i"<<b;
            }
            else{
                cout<<a<<" + i"<<b<<endl;
            }
        }
        void addition(Complex c1, Complex c2){
            a = c1.a + c2.a;
            b = c1.b + c2.b;
        }
        void subtraction(Complex c1, Complex c2){
            a = c1.a - c2.a;
            b = c1.b - c2.b;
        }
        friend Complex mulComplex(Complex, Complex);
        Complex operator / (Complex c2){
            Complex t;
            t.a = (a*c2.a + b*c2.b)/(c2.a*c2.a + c2.b*c2.b);
            t.b = (b*c2.a - a*c2.b)/(c2.a*c2.a + c2.b*c2.b);
            return t;
        }

};
Complex mulComplex(Complex c1, Complex c2){
    Complex t;
    t.a = c1.a * c2.a - c1.b * c2.b;
    t.b = c1.b * c2.a + c1.a * c2.b;
    return t;
}
int main(){

    Complex c1, c2, ad, sb, mul, div;
    cout<<"Enter first complex number: ";
    c1.readComplexNumber();
    cout<<"\nFirst complex number is :\n";
```

```
    c1.displayComplexNumber();
    cout<<"Enter second complex number: ";
    c2.readComplexNumber();
    cout<<"\nSecond complex number is :\n";
    c2.displayComplexNumber();
    ad.addition(c1, c2);
    cout<<"\nAddition is :\n";
    ad.displayComplexNumber();
    sb.subtraction(c1, c2);
    cout<<"\nSubtraction is :\n";
    sb.displayComplexNumber();
    mul = mulComplex(c1, c2);
    cout<<"\nMultiplication is :\n";
    mul.displayComplexNumber();
    div = c1 / c2;//div = c1.divComplex(c2);
    cout<<"\nDivision is :\n";
    div.displayComplexNumber();
    return 0;
}
```

← RUNTIME POLYMORPHISM →

```
#include <iostream>
using namespace std;

class Base {
    public: void fun() {
        cout << " Base function... " << endl;
    }
};
class Derived : public Base {
    public: void fun() {
```

```
        cout << " Derived function... " << endl;
    }
};
int main() {
    Base *Bp1= new Base ();
    Bp1->fun(); // Base function
    Derived *Dp1= new Derived ();
    Dp1->fun(); // Derived function
    Base *Bp2= new Derived ();
    Bp2->fun(); // Base function
 // Due to the superiority of the base class pointer, the base class function is called instead of the derived class function.
 // This is because the base class pointer can only access the members of the base class, and it does not have knowledge of the derived class members.
 return 0;
}
```

←—- EXAMPLE CODE FOR CREATION OF CLASS ——→

```
#include <bits/stdc++.h>
using namespace std;

// Base Class
class Shape {
protected:
    string name;
public:
    Shape(string n) : name(n) {}
```

```
    virtual ~Shape() {}
    string getName() const { return name; }
};

// 2D Base Class
class TwoDimensionalShape : public Shape {
public:
    TwoDimensionalShape(string n) : Shape(n) {}
    virtual double area() = 0;
    virtual double perimeter() = 0;
};

// 3D Base Class
class ThreeDimensionalShape : public Shape {
public:
    ThreeDimensionalShape(string n) : Shape(n) {}
    virtual double lateralSurfaceArea() = 0;
    virtual double totalSurfaceArea() = 0;
    virtual double volume() = 0;
};

// 2D Derived Classes
class Circle : public TwoDimensionalShape {
    double radius;
public:
    Circle(double r) : TwoDimensionalShape("Circle"), radius(r) {}
    double area() override { return M_PI * radius * radius; }
    double perimeter() override { return 2 * M_PI * radius; }
};

class Rectangle : public TwoDimensionalShape {
    double length, width;
public:
    Rectangle(double l, double w) : TwoDimensionalShape("Rectangle"), length(l), width(w)
{}
    double area() override { return length * width; }
```

```
    double perimeter() override { return 2 * (length + width); }
};

class Square : public TwoDimensionalShape {
    double side;
public:
    Square(double s) : TwoDimensionalShape("Square"), side(s) {}
    double area() override { return side * side; }
    double perimeter() override { return 4 * side; }
};

// 3D Derived Classes
class Cube : public ThreeDimensionalShape {
    double side;
public:
    Cube(double s) : ThreeDimensionalShape("Cube"), side(s) {}
    double lateralSurfaceArea() override { return 4 * side * side; }
    double totalSurfaceArea() override { return 6 * side * side; }
    double volume() override { return pow(side, 3); }
};

class Cuboid : public ThreeDimensionalShape {
    double length, width, height;
public:
    Cuboid(double l, double w, double h) : ThreeDimensionalShape("Cuboid"), length(l),
width(w), height(h) {}
    double lateralSurfaceArea() override { return 2 * height * (length + width); }
    double totalSurfaceArea() override { return 2 * (length * width + width * height + height
* length); }
    double volume() override { return length * width * height; }
};

class Sphere : public ThreeDimensionalShape {
    double radius;
public:
    Sphere(double r) : ThreeDimensionalShape("Sphere"), radius(r) {}
```

```
    double lateralSurfaceArea() override { return 4 * M_PI * radius * radius; }
    double totalSurfaceArea() override { return 4 * M_PI * radius * radius; }
    double volume() override { return (4.0 / 3.0) * M_PI * pow(radius, 3); }
};

int main() {
    Circle c(7);
    cout << c.getName() << " Area : " << c.area() << " , Perimeter : " << c.perimeter() << "\n";

    Cuboid cb(6, 4, 5);
    cout << cb.getName() << " LSA : " << cb.lateralSurfaceArea()
         << " , TSA : " << cb.totalSurfaceArea()
         << " , Volume : " << cb.volume() << "\n";

    return 0;
}
```

🥺 PART – 5

💻 Object-Oriented Programming & Inheritance in C++

Mastering Class Hierarchies, Access Modifiers, Constructor Invocation Order, and Object Lifecycles

---

💡 At a Glance: The Blueprint & Extension Analogy

Inheritance allows a derived class (*specialized blueprint*) to absorb data members and member routines of an existing base class (*fundamental blueprint*). Instead of duplicating boilerplate code (getters/setters, state properties), derived classes extend functionality while maintaining strict **encapsulation**.

When instantiating a derived class, **constructors execute top-down** (Base class constructor initializes first, followed by the Derived class constructor). **Destructors execute bottom-up** in reverse order.

🏗️ 1. Core Concepts & Architectural Motivation

- **Topic:** Hierarchical Inheritance, Visibility Scopes, Constructor Chaining, and Destructors in C++.
- **Key Architectural Principle: Code Reusability & Zero Duplication**:
    - **Base Class Abstraction:** Encapsulates foundational entity state (Person).
    - **Derived Class Extension:** Specializes identity without redeclaring existing properties (Student, Faculty, Derived).
    - **Maintenance Efficiency:** Modifying a base method automatically propagates improvements to all sub-classes.

🔑 2. Modes of Inheritance & Accessibility Matrix

When inheriting from a base class, the specified derivation access mode establishes an upper bound on member visibility within the derived scope:

-

Base Member Access	Public Inheritance (: public)	Protected Inheritance (: protected)	Private Inheritance (: private)	Accessibility Outside Hierarchy
public	Remains public	Becomes protected	Becomes private	Yes (via Public only)
protected	Remains protected	Becomes protected	Becomes private	No (Internal to hierarchy)
private	Inaccessible directly	Inaccessible directly	Inaccessible directly	No (Base methods only)

🛡️ Why Encapsulation Matters: The Whiteboard Breakdown

- **Private Members (int x):** Marked with strict refusal (X X X). Private members of a base class are **never directly accessible** in derived scopes (Derived1, Derived2, or Derived3).
- **Protected Members (int y):** Serve as the bridge for inheritance—accessible to family classes, hidden from outside callers.
- **Public Members (int z):** Exposed interfaces for world interaction.
  ○

```
// ========================================

// File:    InheritanceModes.cpp

// Topic:   Access Specifiers under Derivation

// ========================================



class Base {

private:

    int x;   // Inaccessible in ANY derived class (X X X)
```

```
protected:
    int y;   // Family accessible
public:
    int z;   // Public interface
};


// --- Mode 1: Private Inheritance ---
class Derived1 : private Base {
    // x: Inaccessible
    // private:   y, z  (both demoted to private)
    // protected: (none)
    // public:    (none)
};
```

```
// --- Mode 2: Protected Inheritance ---

class Derived2 : protected Base {

    // x: Inaccessible

    // private:   (none)

    // protected: y, z  (both capped at protected)

    // public:    (none)

};




// --- Mode 3: Public Inheritance ---

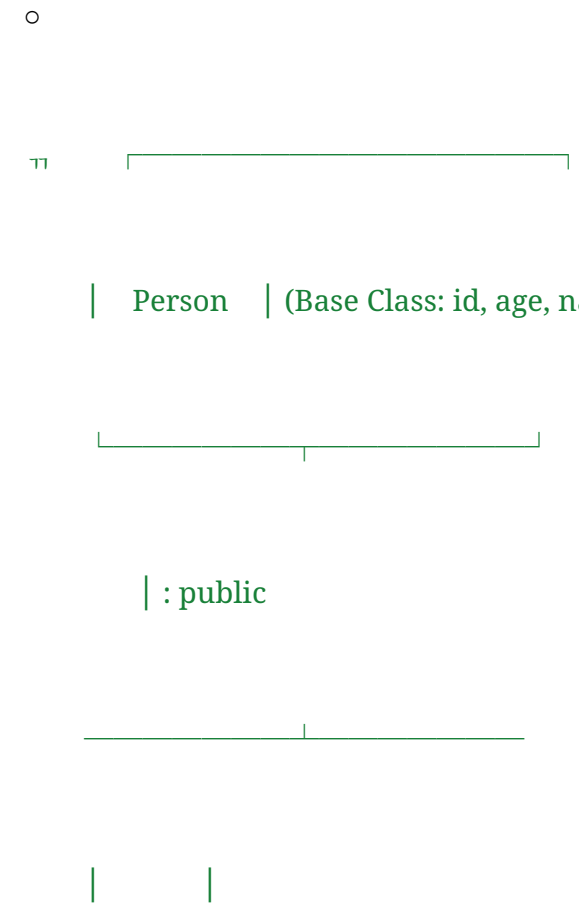
class Derived3 : public Base {

    // x: Inaccessible

    // private:   (none)
```

```
    // protected: y      (remains protected)
    // public:    z      (remains public)
};
```

🏛️ 3. Class Design & Hierarchy Breakdown





1. Base Class: Person

- **Data Members:**
  - private: long id; — Unique personal ID (Encapsulated)
  - protected: int age; — Age property (Accessible to derived classes)
  - public: string name; — Public name property
- **Member Functions:**
  - Dedicated input setters: setId(), setAge(), setName()
  - Formatted output getters: getId(), getAge(), getName()

2. Derived Class: Student : public Person

- **Inherited Members:** Reuses name, age, and id accessors from Person.
- **Specialized Data Members:**
  - private: double gpa; — Grade Point Average
  - public: string rno; — Academic Roll Number
- **New Member Functions:** setRollNumber(), setGpa(), getRollNumber(), getGpa()

3. Derived Class: Faculty : public Person

- **Inherited Members:** Reuses name, age, and id accessors from Person.
- **Specialized Data Members:**
    - private: int sal; — Faculty salary
    - protected: string research; — Research domain
    - public: string fid; — Faculty employee ID
    - public: string course; — Course assigned
- **New Member Functions:** setFid(), setSal(), setResearch(), setCourse(), and matching getters.

⚡ 4. Constructor & Destructor Execution in Inheritance

🔄 Order of Invocation Rule

- **Constructors (Top-Down):** When a derived class instance is created, the **Base class constructor runs first**, followed by the **Derived class constructor**.
- **Destructors (Bottom-Up):** When an instance goes out of scope, the **Derived class destructor runs first**, followed by the **Base class destructor**.
- **Parameterized Constructor Chaining:** To pass parameters to a base class constructor, the derived class constructor explicitly calls the base constructor using an **initialization list**: Derived(int a_val, int b_val) : Base(a_val) { this->b = b_val; }

```
//
================================================================
=============

// File:        ConstructorInheritanceDemo.cpp

// Topic:       Order of Constructor & Destructor Invocations
in C++

//
================================================================
=============



#include <iostream>

using namespace std;



// --- Base Class Definition ---

class Base {

protected:
```

```
    int a;



public:

    // 1. Default Constructor

    Base() {

        a = 0;

        cout << "Base class default constructor executed.\n";

    }



    // 2. Parameterized Constructor

    Base(int a) {

        this->a = a;

        cout << "Base class parameterized constructor executed
(a = " << a << ").\n";
```

```
}



void setA() {

    cout << "Enter value of a in Base: ";

    cin  >> a;

}



void getA() const {

    cout << "Value of a in Base is: " << a << "\n";

}



// Destructor

virtual ~Base() {
```

```
        cout << "Base class destructor executed.\n";
    }
};

// --- Derived Class Definition (Public Derivation) ---
class Derived : public Base {
private:
    int b;

public:
    // 1. Default Constructor (Base() automatically called first)
    Derived() : Base() {
        b = 0;
```

```
        cout << "Derived class default constructor executed.
\n";

    }



    // 2. Parameterized Constructor (Explicitly routes a_val to
Base(a))

    Derived(int a_val, int b_val) : Base(a_val) {

        this->b = b_val;

        cout << "Derived class parameterized constructor
executed (b = " << b_val << ").\n";

    }



    void setB() {

        cout << "Enter value of b of Derived class: ";

        cin  >> b;
```

```
    }



    void getB() const {

        cout << "The value of b in Derived class is: " << b <<
"\n";

    }



    // Destructor

    ~Derived() override {

        cout << "Derived class destructor executed.\n";

    }

};

ಠ
```

⚙️ 5. Member Count & Method Metric Matrix

Class Entity	Inherited Accessible Data	Specialized Data Members	New Methods Defined	Total Invokable Methods
Person	0	3 (1 private, 1 protected, 1 public)	6 (3 setters, 3 getters)	6

Class Entity	Inherited Accessible Data	Specialized Data Members	New Methods Defined	Total Invokable Methods
Student	2 (age, name)	2 (1 private, 1 public)	4 (2 setters, 2 getters)	10 (4 new + 6 inherited)
Faculty	2 (age, name)	4 (1 priv, 1 prot, 2 pub)	8 (4 setters, 4 getters)	14 (8 new + 6 inherited)
Base (Demo)	0	1 (protected a)	4 (2 ctors, setA, getA)	4

Class Entity	Inherited Accessible Data	Specialized Data Members	New Methods Defined	Total Invokable Methods
Derived (Demo)	1 (protected a)	1 (private b)	4 (2 ctors, setB, getB)	6 (4 new + 2 inherited)

🖥️ 6. Implementation: Complete Execution Driver

-

ㄱㄱ//
==========
==========

```cpp
// File:        AcademicInheritance.cpp

// Topic:       C++ Multi-Class Inheritance & Scope Resolution

// Compiler:    C++17 / C++20 Standard

//
================================================================
==============




#include <iostream>

#include <string>



using namespace std;



//
================================================================
==============
```

```
// 1. BASE CLASS: Person

//
==========================================================================
=============

class Person {

private:

    long id;



protected:

    int age;



public:

    string name;
```

```
void setId() {

    cout << "Enter Id:\n";

    cin  >> id;

}




void setAge() {

    cout << "Enter Age:\n";

    cin  >> age;

}




void setName() {

    cout << "Enter Name:\n";

    cin.get();
```

```
    getline(cin, name);

}



void getId() {

    cout << "\nId is : " << id;

}



void getAge() {

    cout << "\nAge is : " << age;

}



void getName() {

    cout << "\nName is : " << name;
```

```
    }

};



//
================================================================
==============

// 2. DERIVED CLASS: Student (Public Inheritance)

//
================================================================
==============

class Student : public Person {

private:

    double gpa;



public:
```

```
string rno;



void setRollNumber() {

    cout << "Enter Roll number: \n";

    cin  >> rno;

}



void setGpa() {

    cout << "Enter GPA: \n";

    cin  >> gpa;

}



void getRollNumber() {
```

```
        cout << "\nRoll Number is : \n" << rno;

    }



    void getGpa() {

        cout << "\nGPA is : \n" << gpa;

    }

};



//
================================================================
=============

// 3. DERIVED CLASS: Faculty (Public Inheritance)

//
================================================================
=============
```

```
class Faculty : public Person {

private:

    int sal;



protected:

    string research;



public:

    string fid;

    string course;



    void setFid() {

        cout << "\nEnter Faculty Id:\n";
```

```
    cin  >> fid;

}



void setSal() {

    cout << "\nEnter Faculty salary:\n";

    cin  >> sal;

}



void setResearch() {

    cout << "\nEnter Faculty Research Area:\n";

    cin.get();

    getline(cin, research);

}
```

```
void setCourse() {

    cout << "\nEnter Course taught by Faculty:\n";

    cin.get();

    getline(cin, course);

}



void getFid() {

    cout << "\nId is : " << fid;

}



void getSal() {

    cout << "\nSalary is : " << sal;
```

```
    }



    void getResearch() {

        cout << "\nResearch Area is : " << research;

    }



    void getCourse() {

        cout << "\nCourse taught : " << course;

    }

};



//
==========================================================================
=============
```

```
// 4. MAIN DRIVER ROUTINE

//
================================================================
=============

int main() {

    cout << "=== 1. TESTING CONSTRUCTOR INVOCATION ORDER
===\n";

    // Base object creation

    Base x;                // Calls Base default constructor

    x.getA();



    Base y(20);            // Calls Base parameterized
constructor

    y.getA();
```

```
    // Derived object creation (Default)

    Derived d1;           // Calls Base() FIRST, then Derived()

    d1.getB();




    // Derived object creation (Parameterized)

    Derived d2(30, 60);   // Calls Base(30) FIRST, then
Derived(60)

    d2.getA();            // Inherited method from Base

    d2.getB();            // Specialized method in Derived




    cout << "\n=== 2. TESTING ACADEMIC CLASS HIERARCHY ===\n";

    // --- Person Instance ---

    Person p1;

    cout << "Enter details of a person p1:\n";
```

```
p1.setId();

p1.setName();

p1.setAge();

cout << "\nPerson p1 details are:";

p1.getId();

p1.getName();

p1.getAge();



// --- Student Instance ---

Student s1;

cout << "\n\nEnter details of Student s1:\n";

s1.setRollNumber();

s1.setName();
```

```
s1.setGpa();

s1.setAge();

cout << "\nStudent s1 details are:";

s1.getRollNumber();

s1.getName();

s1.getGpa();

s1.getAge();



// --- Faculty Instance ---

Faculty f1;

cout << "\n\nEnter details of Faculty f1:\n";

f1.setFid();

f1.setName();
```

```
    f1.setSal();

    f1.setAge();

    f1.setResearch();

    f1.setCourse();

    cout << "\nFaculty f1 details are:";

    f1.getFid();

    f1.getName();

    f1.getSal();

    f1.getAge();

    f1.getResearch();

    f1.getCourse();



    cout << "\n\nExiting main (Destructors execute in reverse
order)...\n";
```

```
    return 0;

}
```

ठ

📌 7. Quick Exam Checklist & Review

- **Constructor Execution Order:** When an object of a derived class is initialized, the base class constructor **always runs first** before the derived class constructor body.
- **Destructor Execution Order:** Destructors execute in the **exact reverse order** (Derived destructor runs first, then Base destructor).
- **Parameterized Base Forwarding:** Use the member initialization list syntax `Derived(int a, int b) : Base(a) { ... }` to pass parameters to the base constructor.
- **Inheritance Visibility Rule:** Under public inheritance, `public` remains public, `protected` remains protected. Base `private` members remain strictly inaccessible.
- **Stream Buffer Hygiene:** Always insert `cin.get()` before `getline()` when switching from `cin >>` to line streams to discard the trailing newline character.